Exercise: Incremental Load

Explore incremental loading methods by building an ETL load step that imports daily log activity data into PostgreSQL. Learn to create tables, write loading functions with the COPY command, and automate deletion of data older than one month to optimize storage and performance.

In this exercise, we'll practice using the `COPY` command to incrementally load data. In incremental loading, we only load the recent changes in the source into the destination repository. As always, the logic by which we do it depends on the business requirement and context.

Example

As data engineers working for a retail company, we're tasked with creating an ETL pipeline for loading customers' log activity data from a transactional database into a PostgreSQL database for analysis.

According to the business requirements, new batches of data should be loaded at the end of each day. Also, to save storage costs, the PostgreSQL database should only contain data from the last month. After creating the extract and transform steps of our pipeline, we'll now focus on the load step. The extract and transform steps results are a CSV file containing the log activity data for the last day.

Our task is to create a table for storing the log activity data and write a PostgreSQL function that does the following:

Ask

1. Load the data from the CSV file into the PostgreSQL database.

2. Check if there's outdated data in the database (data that is older than one month) and delete it.

Here's a sample of the log activity data:

index.md Markdown

```
id,user_id,activity,date
1,1,login,2023-06-01:00:00:00
2,2,login,2023-06-01:00:00:05
3,3,login,2023-06-01:00:01:10
4,1,logout,2023-06-01:00:01:15
5,4,login,2023-06-01:00:01:20
6,2,logout,2023-06-01:01:01:20
7,5,register,2023-06-01:01:01:25
8,6,register,2023-06-01:01:01:30
9,7,mouseclick,2023-06-01:01:01:35
10,8,mouseclick,2023-06-01:02:01:40
```

Sample data

> **Note:** Fill in the missing code in the code widgets to complete the exercise.

Task 1: Create the `log_activity` table

First, create a table in PostgreSQL to store the log activity data. The table requirements are:

1. `log_id (int)`
2. `user_id (int)`
3. `activity (varchar(30))`
4. `date (timestamp)`

Ask

Also, the primary key is the combination of `log_id` and `user_id`.

main.sql PostgreSQL

```
create table log_activity(
    <<InsertHere>>
    <<InsertHere>>
    <<InsertHere>>
    <<InsertHere>>
    primary key(log_id, user_id)
);
```

Run

Creating a table

Task 2: Create the `load_data()` function

As mentioned, the loading function will perform two operations: loading the data from the CSV file into the PostgreSQL database and deleting outdated data. For this task, complete the code for loading the CSV file.

main.sql PostgreSQL

```
CREATE OR REPLACE FUNCTION load_data()
RETURNS boolean
LANGUAGE plpgsql AS $$
BEGIN

    -- 1. Load CSV file
    raise notice 'Inserting rows';
    <<InsertHere>
    from '/usercode/tmp/log_activity_data.csv'
    <<InsertHere>
    <<InsertHere>

    raise notice 'Done!';
```

Ask

Run

Task 3: Delete outdated data

The last task is to add a script to the function that deletes data older than one month. This will help keep the number of rows of this table in check.

main.sql PostgreSQL

```
CREATE OR REPLACE FUNCTION load_data()
RETURNS boolean
LANGUAGE plpgsql AS $$
BEGIN

    -- 1. Load CSV file
    raise notice 'Inserting rows';
    copy log_activity
    from '/usercode/tmp/log_activity_data.csv'
    delimiter ','
    CSV header;

    -- 2. Delete outdated data
```

Run

Solution codes

Here are the solution codes for the above tasks:

Task 1: Create the `log_activity` table

main.sql PostgreSQL

```
create table log_activity(
    log_id int,
    user_id int,
    activity varchar(30),
    date timestamp,
    primary key(log_id, user_id)
```

Ask

```
);
```

Run

Solution code: Create the log_activity table

- **Lines 2–5**: We define the table}s columns and respective data types.
- **Line 6**: We set the table's primary key as the combination of `log_id` and `user_id`.

Task 2: Create the `load_data()` function

main.sql PostgreSQL

```
CREATE OR REPLACE FUNCTION load_data()
RETURNS boolean
LANGUAGE plpgsql AS $$
BEGIN

    -- 1. Load CSV file
    raise notice 'Inserting rows';
    copy log_activity
    from '/usercode/tmp/log_activity_data.csv'
    delimiter ','
    CSV header;

    raise notice 'Done!';
```

Run

Solution code: Create the load_data() function

- **Lines 7–11**: We use the `COPY` command to load data from `'/usercode/tmp/log_activity_data.csv'` to the `log_activity` table.

Ask

Task 3: Delete outdated data

main.sql PostgreSQL

```
CREATE OR REPLACE FUNCTION load_data()
RETURNS boolean
LANGUAGE plpgsql AS $$
BEGIN

    -- 1. Load CSV file
    raise notice 'Inserting rows';
    copy log_activity
    from '/usercode/tmp/log_activity_data.csv'
    delimiter ','
    CSV header;

    -- 2. Delete outdated data
```

Run

Solution code: Delete outdated data

- **Lines 15–16**: From the `log_activity` table, we delete all records with a date earlier than last month.

Mark As Completed

Ask